

39 Box Stage Model (Version 2.0)

Michael Waldon

July 2025

Contents

1	Quick start	1
2	Introduction	3
2.1	Model need	3
2.2	A.R.M. Loxahatchee National Wildlife Refuge	5
2.3	History of model development	7
2.4	Major assumptions and limitations	10
3	Model Description	12
3.1	Conceptual framework	12
3.2	Data sources	13
3.3	Water volume and stage simulation	13
3.3.1	Cell water budget	13
3.3.2	Inflow and outflow	14
3.3.3	Water flow between compartments	15
3.3.4	Link velocity	15
3.4	Overall Refuge Water budget	16
3.4.1	Inflow	17
3.4.2	Outflow	17
3.5	Temporal scale	23
3.6	Model Structure	23
3.7	Time, day, and date	24
3.8	Parameters	24

3.9	Timeseries and geometric data	24
3.9.1	Input data types and sources	26
3.9.2	R dataset creation	27
3.10	Model Implementation in R	27
3.10.1	Rationale for porting to R	27
3.10.2	Key R program libraries used	28
3.10.3	File structure and organization	28
3.11	Model calibration	30
4	Running the Model	33
4.1	Model package description	33
4.2	System requirements and installation	34
4.3	Output	34
4.4	Axillary Scripts for Output Visualization	35
4.4.1	Particle Tracking	35
4.4.2	Plotting contours of marsh properties	36
4.4.3	Plotting canal properties	39
4.4.4	Additional location-related functions	42
4.5	Workflow	46
5	References	50
A	Glossary of Terms	53
B	Extending the model timeseries	55
B.1	Evapotranspiration (ET)	55
B.2	Rainfall	56
B.3	USGS daily average stage data	56
B.4	Inflow/Outflow	58

Abstract

This user manual describes a new implementation developed in the R scripting language of the 39-Box (compartment) stage model, which was developed from an earlier version. The stage model simulates water stage and flow in the Arthur R. Marshall Loxahatchee National Wildlife Refuge. A companion 39-Box Mass model is under development, which will simulate the concentrations of constituents, including chloride, sulfate, and phosphate.

The 39-Box Stage Model predicts the stage (water surface elevation), water depth, and exchange flow in the Refuge Canal and wetlands. These stage predictions are generated from a series of compartmental water budgets. The model divides the Refuge into 39 boxes (also termed compartments, nodes, or tanks), and water movement into and out of each box is estimated to give the rate of volume increase or decrease for each box. The boxes were first delineated in the 9-Box Model based on available water quality monitoring data (Wang et al., 2008) and further divided into the 39-Box design to provide greater insight into the impacts of water management alterations.

The model code, datasets, and documentation are available from the GitHub repository <https://github.com/MWaldon/Loxahatchee-39-Box-Model>

Chapter 1

Quick start

To productively apply the 39-Box model, users need to have a basic familiarity with programming in R. In this quick-start section, it is assumed that the model scripts will be executed in the R-Studio environment. To run the model, both R and R-Studio must be installed.

To begin using the model, users may follow these steps:

1. Obtain the distribution compressed file *39BoxStageV20.zip*) containing the model folders and files. This file is available from the GitHub repository github.com/MWaldon/Loxahatchee-39-Box-Model
2. In R-Studio create a new project with an appropriate name. Select any desired path on your computer for the project home directory.
3. Copy the folders from the distribution zip file to the new project home directory.
4. Open the file `runStage.R` from the installed `Rscript` folder
5. In the dropdown menu "Sessions/Set Working Directory" choose "To Source File Location." This points the working directory path to the `Rscript` folder.
6. In the editor window click "Source."

7. Move to the console window (click in the Console window or press Ctrl+2) and then, in answer to the prompt, hit enter to accept the default filename.
8. Click the "Plots" tab (or press Ctrl+6).
9. A plot should open and plot one point for each day of the model run.
10. When the model has completed a series of plots will be drawn.

See the Workflow section (4.5) for more information on how to run the model.

Chapter 2

Introduction

2.1 Model need

The Arthur R. Marshall Loxahatchee National Wildlife Refuge (Refuge) has been impacted by excessive levels of phosphorus and other contaminants in stormwater inflows. Major restoration projects by the South Florida Water Management District (SFWMD) and the US Army Corps of Engineers have been implemented to reverse these impacts on the Everglades. In addition to reducing the concentration of nutrients and contaminants, these projects reduced the total inflow and altered the inflow patterns.

As the restoration of the Refuge continues, quantitative methods are needed to predict the impacts of existing and proposed management changes by quantifying changes in the quantity and quality of water in the refuge caused by restoration efforts. The 39-Box Model was developed to meet many of these needs. The 39-Box Stage Model predicts water volumes, stages (*i.e.* water surface elevations), water depths, and flows over the Refuge. A companion model, the 39-Box Mass Model, is in development. The mass model is linked to the output of the stage model and predicts the mass and concentrations of the selected constituents. These 39-Box models delineate 39 compartments that overlay the refuge canal and marsh (Figure 2.1).

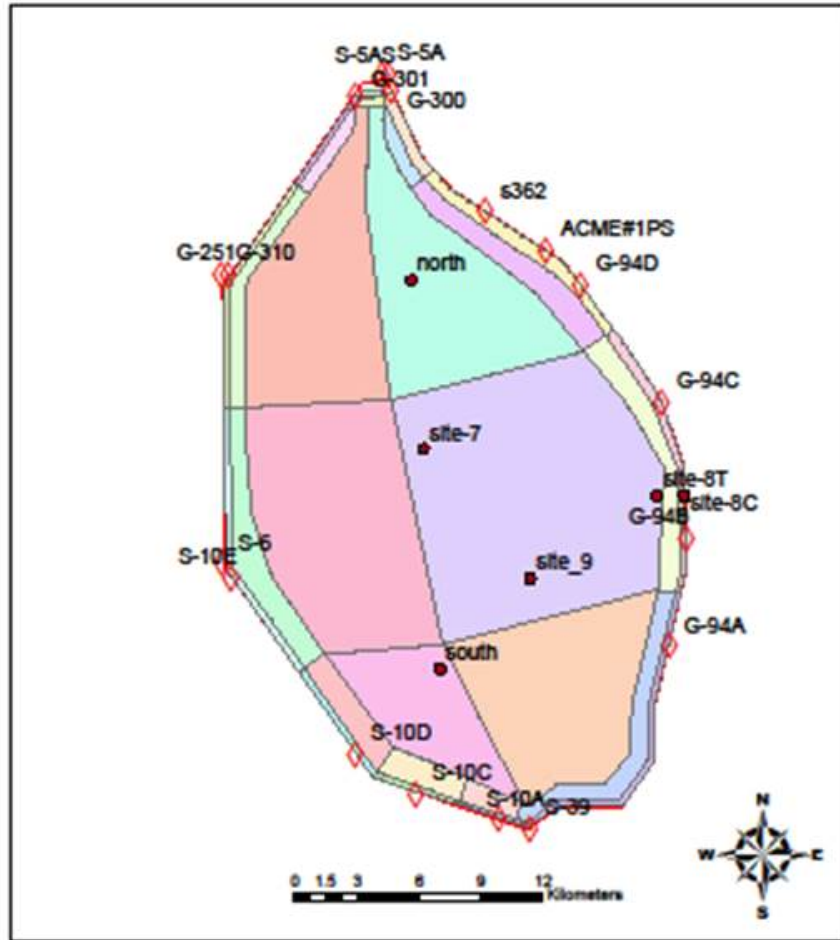


Figure 2.1: The arrangement of the the 39 model Compartments is shown along with Refuge stage gauges and inflow/outflow structures (Bazgirkhoob et al., 2015).

This manual details a new implementation of the 39-Box (compartmental) Stage Model. This implementation is a code translation to the R scripting language from the earlier version of the 39-Box Model that was developed using the Berkeley-Madonna simulation environment. That earlier version developed over a decade from the Refuge 9-Box Model (Wang *et al.*, 2012) and earlier SRSM (Meselhe *et al.*, 2009) and RWQM (Meselhe *et al.*, 2007a, 2007b) models.

This manual aims to provide users with an understanding of both the theory and implementation of the stage model. The objective is to provide users with the ability to implement simulations of alternative water management scenarios for the refuge. In addition to a basic understanding of R programming, model users should have a general familiarity with the Refuge’s geography, hydrologic features, and water quality. For more information, users are directed to the Refuge Comprehensive Conservation Plan (USFWS 2000).

2.2 A.R.M. Loxahatchee National Wildlife Refuge

The Arthur R. Marshall Loxahatchee National Wildlife Refuge was established in 1951 in Palm Beach County, Florida, USA. The Refuge covers an area of 145,188 acres (226 square miles) and includes Water Conservation Area 1 (WCA-1). WCA-1 is one of three Florida Water Conservation Areas which are remnants of the Everglades wetland ecosystem that has been called the River of Grass.

The refuge protects a diverse mosaic of habitats, including sawgrass, cattail, sloughs, and tree island communities. This expansive protected wetland is a crucial habitat for a wide variety of wildlife. It supports more than 250 species of birds, 60 species of reptiles and amphibians, 40 species of butterflies, and 20 species of mammals. Visitors to the refuge can frequently observe alligators, bobcats, white-tailed deer, and various bird species, including the federally endangered snail kite (USFWS, 2000).

Prior to development, natural water flow in the Refuge moved in a generally north-to-south direction, flowing through shallow sloughs and as sheetflow over the marsh. In contrast, the perimeter of the WCA-1 wetland is currently encircled and isolated by a borrow canal and levee. Consequently, water flow over the wetland now begins and ends as it flows over the perimeter canal banks.

All inflows and outflows between the perimeter canal and the surrounding agricultural and urban areas are controlled by water control gates and pumps. All inflows and outflows are monitored, and daily discharge monitoring records are archived and made available by the South Florida Water Management District (SFWMD). The availability of this monitoring is essential for accurate model development.

Water stage and movement are important factors controlling the plant communities in the Refuge (Figure 2.2), and these factors are controlled by the water sources and losses in the Refuge (Figure 3.2). Seasonal patterns of water stage determine the hydroperiod of the marsh, and water movement transports plant nutrients throughout the Refuge.

The Refuge is a source of water retention and supply for the surrounding area. Downstream Everglades wetlands, including other water conservation areas and the Everglades National Park, depend on the Refuge for part of their water supply. In addition to this ecological value, the Refuge is an important part of the water supply and flood management system in South Florida. During heavy rain, stormwater may be pumped directly into the Refuge or enter the Refuge indirectly after passing through stormwater treatment areas. During the dry season, the Refuge provides a supply of water to the Southeast Florida coast to prevent saltwater intrusion into surface or groundwater, which are the sources of water used for agriculture and municipal water supply in South Florida.

The US Army Corps of Engineers has the responsibility of balancing the needs for water in controlling flow out of the Refuge through the S-10 gates (Figure 3.3). To meet this responsibility, the Corps, in consultation with all

affected parties, developed a water regulation schedule for the Refuge. The current regulation schedule took effect in 1995 (Figure 3.6).

2.3 History of model development

Investigators have worked to develop Refuge computer models since the late 1970s (Arceneaux, *et al.* ., 2007). Notably, Richardson *et al.* . (1990, 1993) recognized the need for a stage model to understand how Refuge water management impacts hydroperiod and vegetation. They developed a GIS-linked hydrologic model of the Refuge by adapting an earlier Everglades-wide water model. Their Refuge model defined 750 square grid cells measuring 3000×3000 ft to represent the Refuge marsh area.

In the early 2000s, the US Fish and Wildlife Service was funded to develop an enhanced monitoring and modeling program for the Refuge. The modeling effort of this program developed at the University of Louisiana under the mentorship of the project principal investigator, Dr. Ehab Meselhe. The technical management of the modeling effort was led by USFWS senior hydrologist Dr. Michael Waldon. This modeling project developed two complementary modeling approaches. One approach applied a high spatial resolution commercial hydrodynamic and water quality model, Mike-Flood (Chen, *et al.* ., 2009.). The second approach applied simpler compartmental modeling to simulate the Refuge water quantity and quality. The 39-Box Model was developed as part of this compartmental modeling effort.

Compartmental modeling in this project transitioned through various computer programs and modeling platforms. Version 1 of the model (Arceneaux *et al.*, 2007; Meselhe *et al.*, 2007a, 2007b) was implemented as a daily water budget using a Microsoft Excel workbook for water volume modeling, and the EPA WASP model simulating chloride and total phosphorus concentrations. Versions 2 and 3 of the SRSM were transitional and are no longer available. SRSM version 4.0 (Meselhe *et al.* . 2009; Roth and Meselhe, 2010) was developed using the commercial numerical differential equation solver package of Berkeley Madonna (Macey *et al.* . 2000). It allows greater clar-

ity in coding, supports shorter time steps, obviating the need for the ad hoc procedures of Version 1.0, and provides a well-documented user interface. It simulates the water budget, regulation schedule-related discharges, and constituent dynamics for chloride (Cl), sulfate (SO₄), and total phosphorus (TP) over the period from January 1, 1995, to June 30, 2009.

Wang et al. (2008) applied cluster analysis of chloride, total phosphorus, sulfate, and calcium concentrations measured at sites distributed throughout the Refuge to objectively determine the number of compartments and spatially delineate compartments with similar features within the Refuge. This delineation led to a 9-Box (compartment) model structure (Wang et al., 2012).

The 9-Box model was calibrated and performed well when compared to historic observations. However, there was concern that the model might not perform well when simulating certain management scenarios. This concern led to further disaggregation of the 9-Box model into the 39-Box configuration applied in the current model.

As with the SRSB and 9-Box model, the 39 Box model is based on a link-node conceptualization. Each node in a link-node model has the state variables of water volume, as well as constituent masses for concentration modeling. Stage, depth, and constituent concentrations are calculated from the state variables of compartment volume and total constituent masses.

Flow of water and transport of constituent mass (mass flux) occur between nodes through links. A link defines a flow path between two adjacent nodes, and serves to transfer water volume between its end nodes. Each link flow originates from the end node with the higher stage, and discharges into the lower-stage end node. For each link, an upstream and downstream node is arbitrarily defined so that positive flow moves in the upstream-to-downstream direction and negative flow moves from downstream to upstream. That is, link flow is positive when the upstream node has a higher stage than the downstream node, and link flow is negative when the downstream node has a higher stage than the upstream node.

In programming terminology, links and nodes (compartments) are objects defined by their parameters (also called properties) and processes (also called methods). Links have properties of length, width, depth, and water surface slope, and processes of flow rate. Nodes have the properties of volume, depth, and surface area, and have the process of volume accumulation through the integral of net inflow. Each of these models (SRSM version 1, SRSM version 4, 9-Box Model, and 39-Box Model) has specific advantages and limitations. The models were designed to be a complementary suite. As such, the 39-Box Model is not intended to replace the other models, but instead to provide additional options for Refuge modeling applications. The 39-Box Model provides the highest spatial resolution. However, this higher resolution results in longer runtimes and a more complex output dataset.

2.4 Major assumptions and limitations

To avoid misinterpretations and erroneous conclusions from the model results, it is essential that model users bear in mind the major model assumptions and limitations associated with this model. The input data used in this model are archived as daily averaged values. This includes stage, flow, precipitation, and evapotranspiration data. This averaging may limit the ability of the model to accurately simulate some rare events, but this temporal averaging should not significantly impact longer-term accuracy. Each model cell represents an area rather than a single point, and each cell uses a single soil elevation. Some site-specific characteristics may need to be considered when comparing model results with data from monitoring sites. Area-averaged precipitation was used to develop a daily rainfall depth time series. Loss of water to groundwater (termed seepage loss or recharge) is modeled as proportional to cell stage minus a constant stage representing water stage outside the refuge. This assumption may lead to an underestimation of water loss during drought conditions. Discharge of regulatory water releases is adjusted daily in response to canal water stage. Historic releases generally follow this pattern but are dependent on the manual operation of gated control structures, which could be affected by the availability

of personnel and other contingencies.

Chapter 3

Model Description

3.1 Conceptual framework

The refuge is compartmentalized into 39 compartments (also termed boxes, nodes, or cells), and each is classified as canal or marsh. The compartment surface areas are assumed constant (Table 1), and cover all of the marsh area totaling 4.03 millionm^2 and the canal area totaling $560.02 \text{ millionm}^2$. Mapping of the cell boundaries was based on results of a cluster analysis (Wang, *et al.*, 2008) The link-node diagram of the refuge model is presented in Figure 3.11. Compartments 1-11 represent the canal cells and compartments 12-39 represent the marsh cells (Table 1 and Figure 3.11).

Conceptually, the 39-Box model comprises 39 compartments or nodes and 52 links between nodes (Figure 3.11). The compartments have water volume, depth, and stage or water surface elevation properties. Eleven nodes define the perimeter canal and 28 nodes cover the marsh. Links have flow rate or discharge properties. Each link has a single upstream and downstream node such that positive flow moves from upstream to downstream.

3.2 Data sources

Data from numerous water quality, meteorological, stage, and discharge monitoring stations in and surrounding the refuge were used in model development. Relevant to the stage model, these stations provided measurements of inflow, outflow, water level, water depth, precipitation, and evapotranspiration. Although there are numerous monitoring stations, some model cells in the marsh and canal have no associated water quality or monitoring station. All data in this model version were imported from previous model versions. Figures 3.3, 3.5, 3.12, and 3.13 map the locations of monitoring sites in the Loxahatchee Refuge. Cell soil elevation was estimated from elevations used in the Refuge Mike-Flood model.

3.3 Water volume and stage simulation

The model calculates the derivative of volume by maintaining a water budget for each cell. That is, the rate of increase or decrease, if negative, in cell volume is equal to the sum of all inflows minus the sum of all loss rates. Water depth is then calculated from volume, and stage is calculated by adding depth to the bed elevation (soil elevation) of the cell.

3.3.1 Cell water budget

Cells gain water volume through precipitation and inflow from adjacent linked cells. Canal cells may also receive inflow from structures, including pumps and gates. Water is lost from cells through seepage to groundwater, evapotranspiration, and flow to adjacent cells through links (Figure 3.1).

Water volume in each cell is the state variable simulated by the model. The rate of change (*i.e.* the derivative) of cell water volume is calculated from the difference between the sum of all water flows entering each cell minus the sum of all flows leaving the cell. Water volume is then integrated by the function *radu* which is *deSolve* library of ordinary differential equation (ODE) solvers.

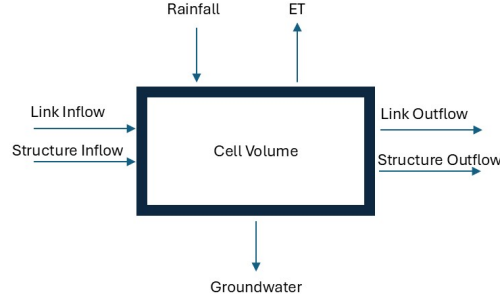


Figure 3.1: Components of water budget for individual cells.

Water depth in each cell is calculated from the relationship between cell depth and cell volume. For marsh cells, depth is simply cell volume divided by a constant cell surface area. For canal cells, the cross-sectional geometry of the canal is used to calculate the relation between cell depth and cell volume. Canal cell depth is measured above the lowest point in the cross-section. Stage (water surface elevation) is calculated by adding cell depth to bottom elevation. All stage and bottom elevations in the model are relative to the NGVD29 datum.

3.3.2 Inflow and outflow

Water flows into and out of the model domain through inflow, outflow, seepage to groundwater, precipitation, and evapotranspiration. Nineteen inflow/outflow structures are represented in the model. The structure inflow timeseries are defined using historical data. Structural outflow may optionally be defined from historic flows or calculated to approximate past structure management based on the water regulation schedule of the Refuge.

3.3.3 Water flow between compartments

Internal flows (m^3/day) between adjacent model cells are termed link flows, which follow paths termed links. Conceptually, flow between cells occurs through links between pairs of adjacent model cells. It may be helpful to think of model cells as tanks and links as pipes between adjacent cells. Links in the 39-Box model are designated by index numbers from 1 to 52 (Figure 3.11). The type of model link is classified into three types: (*linktype* canal-canal (cc), canal-marsh (cm), and marsh-marsh (mm)). The model contains 11 cc, 11 cm, and 30 mm links. Each link has four properties: upstream cell number, downstream cell number, width (m), and radius (m). Cell radius is the estimated length of flow between the upstream and downstream cells.

Link flow is considered positive for flow from the upstream cell to the downstream cell and negative otherwise. Link flows are calculated in the function *Link.Flow.calc(Depth, Stage)* using a power law formula (Kadlec and Knight, 1996).

$$LinkFlow = 10^7 * B * Width * Depth^3 * \frac{Stage_{up} - Stage_{dn}}{Radius} \quad (3.1)$$

Where B is a link-type dependent constant

($cc = 6.97621, cm = 1.13863, mm = 4.55002$),

Width is the width (m) of the shared polygon edge estimated from GIS mapping, *Depth* is the modeled depth of the downstream cell (m), *Stage* is the cell water surface elevation (m), and *Radius* is the flow length of the link estimated as the distance between the centroids of the upstream and downstream cells. Link flows are calculated by the function *Link.Flow.calc*, and net link flow into each of the $n_{cell} = 39$ cells is calculated by the function *link2cell*.

3.3.4 Link velocity

Volumes and link volumetric flow rates are required in model calculations. Link flow velocities (cubic meters/day) are not required for water budget

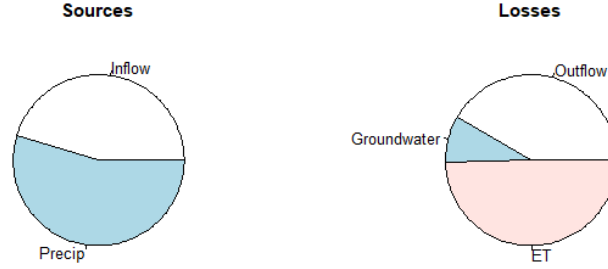


Figure 3.2: The relative importance of total sources and losses of water in the Refuge. This figure was plotted using R script *WaterBalance.R*.

calculations but are calculated by the function *sim.Velocity.calc()* from link flow rates and are stored in a matrix (*sim.Velocity*) in the model output dataset.

3.4 Overall Refuge Water budget

Water flows into and out of the Refuge through water-control structures located on the perimeter canal levees. Precipitation is also a source of water, while water is lost through ET (evaporation and transpiration by plants) and groundwater recharge or seepage (Figure 3.2). Link flows, ET, and loss to groundwater are calculated dynamically and may therefore vary during each day of simulation. Precipitation and all structure inflows and outflows are constant over each day during the simulation.

Inflow/outflow structures

A total of $nstruct = 19$ inflow and outflow structures are modeled (Figure 3.3). In the model code, these 19 water control structures are named:

S39, G94A, G94B, G94C, ACME2,
ACME1, S362, G300, S5AS, S5A,

G301, G310, G251, S6, G338,

S10E, S10D, S10C, S10A.

The percent of total historic inflow and outflow for each of these structures is presented in Figure 3.4. Three structures, *G300*, *G301*, and *S5AS* function for both inflow and outflow.

3.4.1 Inflow

Historic inflows for the period January 1, 1995, through June 30, 2009, are read from an Excel spreadsheet *INFLOW39.xlsx* in the script *make_datasets.R*. This spreadsheet was used to load flows into the model versions. These flows are stored in an R dataset file *39 – Box – Datasets.Rdata*.

The function *Qinfunc(DAY)* returns total inflow discharging into each canal cell on a single day. This function is used by the function *QinExternal.calc* to populate a matrix *QinExternal* with each row containing the daily inflow to each of the *ncanal* canal cells on each day.

Rainfall

In the development of previous models, precipitation, specifically daily rainfall depth, was combined from several weather station sites around the Refuge to create a single time-series dataset. This data is read by the script *make_datasets.R* into the *PET* data frame as *PET\$P*.

3.4.2 Outflow

Historic outflows for the period January 1, 1995, through June 30, 2009, are read from an Excel spreadsheet *OUTFLOW39.xlsx* in the script *make_datasets.R*. This spreadsheet was used to load these flows into the model versions. Additional historic outflows that are not related to the regulation schedule, termed non-regulatory outflows (water supply or major storm operations), are read from an Excel spreadsheet *Regulation39.xlsx*. These flows are stored in the R dataset file *39 – Box – Datasets.Rdata*.

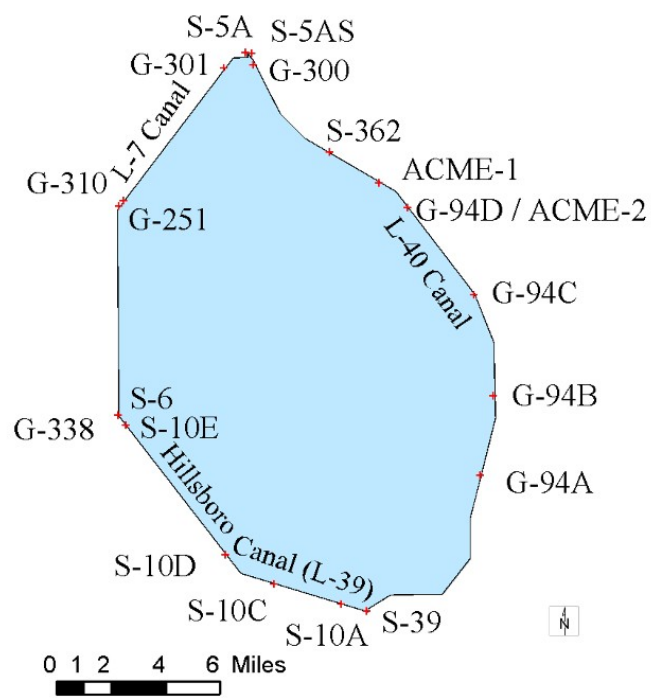


Figure 3.3: Inflow/outflow control structures (Bazgirkhoob *et al.*, 2015).

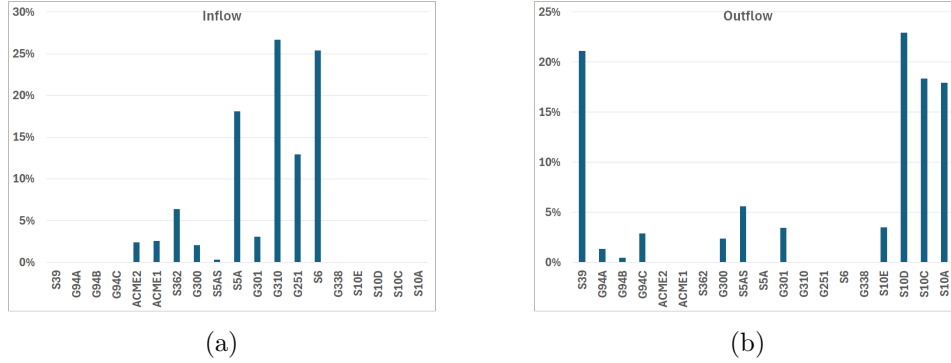


Figure 3.4: Percent of historic total structure inflow (a) and outflow (b) for the period 1/1/1995 to 6/30/2009.

Historic versus calculated outflow

The parameter *CalcQRo* determines whether structural outflows used in a simulation scenario follow historical flows (*CalcQRo* = *FALSE*) or are based on the Refuge Regulation Schedule and calculated from the stage (*CalcQRo* = *TRUE*). Calculated outflows are usually the appropriate option because most modeling scenarios test the impacts of altered water management, water supply, or climate change, and the outflows need to adapt to these changes.

Calculated outflows

Calculated outflow is determined from the difference between the floor of the A1 Zone of the Regulation Schedule (Figure 3.6; USFWS, 2000; Neidrauer, 2004) and the simulated stage at the USGS 1-8C stage gauge. The current regulation schedule was established in May 1995. Therefore, the simulated stage using calculated regulatory outflows and outflows for dates prior to May 1995 should not necessarily track historic values.

The R function *QCalcOutS10(S10Stage, A1)* calculates the total regulatory outflow from ΔS , the difference between its arguments, the simulated stage at the USGS 1-8C gauge (cell 9) and the zone A1 floor stage. The zone A1 floor stage is calculated in function *A1Floor(DAY)* as a function of the

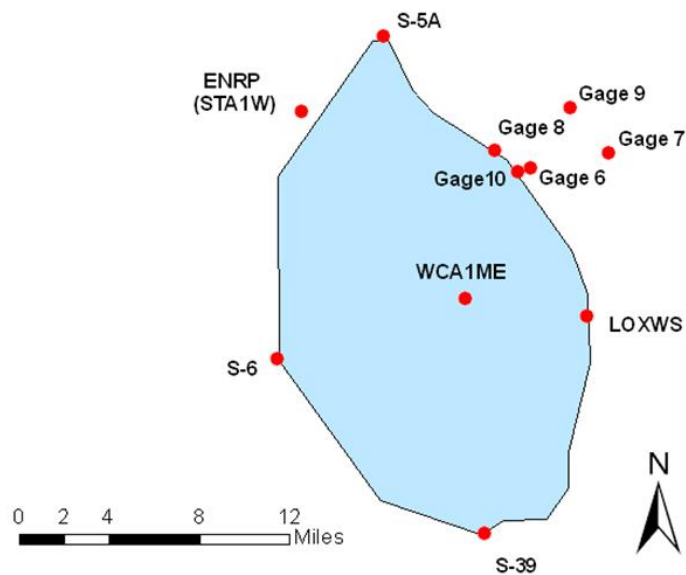
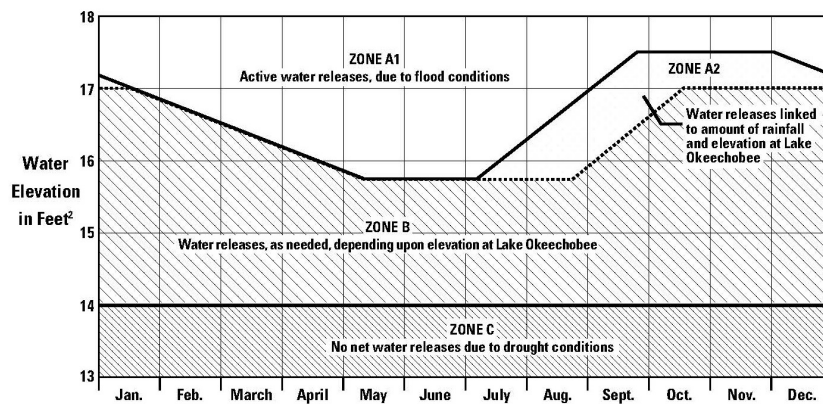


Figure 3.5: Location of rain gauges (Bazgirkhoob *et al.*, 2015).



¹ Established in May, 1995, the water regulation schedule is administered by the U.S. Army Corps of Engineers
² National Geodetic Vertical Datum; Surface water elevation above sea level

Figure 3.6: The Loxahatchee Refuge Regulation Schedule (Arthur R. Marshall Loxahatchee National Wildlife Refuge Comprehensive Conservation Plan(2000)).

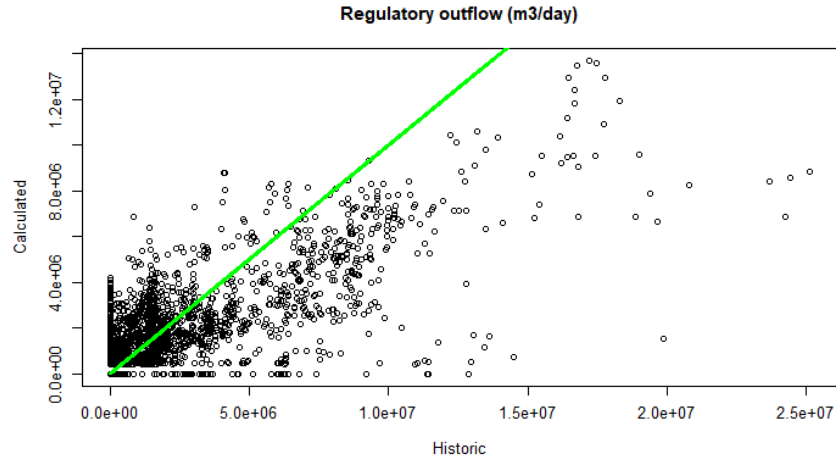


Figure 3.7: Total daily regulatory outflow (m^3/day) from 1/1/1995 to 6/30/2009 are plotted against historic values. The green line indicates a perfect fit.

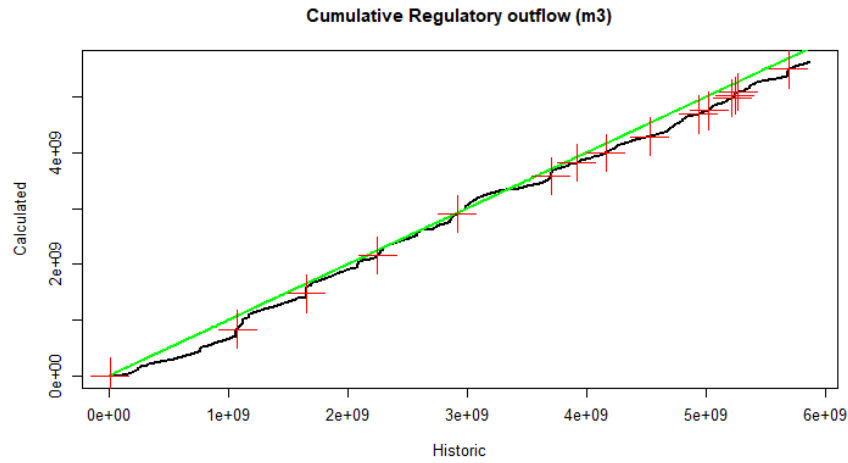


Figure 3.8: Cumulative total of daily regulatory outflow (m^3/day) from 1/1/1995 to 6/30/2009 are plotted against cumulative historic values. The green line is a line of perfect fit. Red crosses are plotted on January 1 of each year.

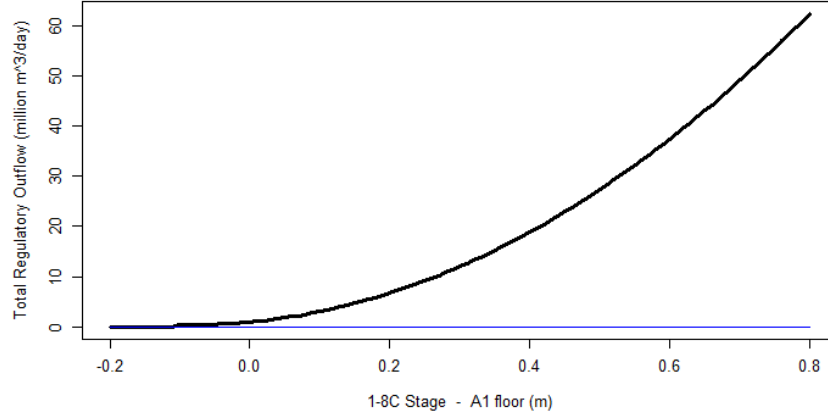


Figure 3.9: Calculated total regulatory outflow (million m^3/day) function (Eq 3.2) plotted against the amount that stage at the USGS 1-8C gauge exceeds the A1 floor stage (m). The blue line is zero outflow.

day of the year (1 to 366). Calculated outflow is zero when $\Delta S < -0.1$. Otherwise

$$Q = -0.9541909 + 12.9026725\Delta S + 79.9679408\Delta S^2 \quad (3.2)$$

where ΔS is the stage at the USGS 1-8C gauge, and Q is the regulatory outflow in millions of cubic meters per day ($10^6 m^3/day$).

Evapotranspiration

Evapotranspiration (ET) represents the movement of water into the atmosphere through the combined processes of evaporation and transpiration. The rate of water loss through ET depends on temperature, wind speed, and water availability. ET is a significant component of Everglades water budgets (German, 2000; Roth and Meselhe, 2010). Potential ET is the rate at which water is lost through ET under completely flooded conditions. The time series of daily potential ET used here was initially downloaded from

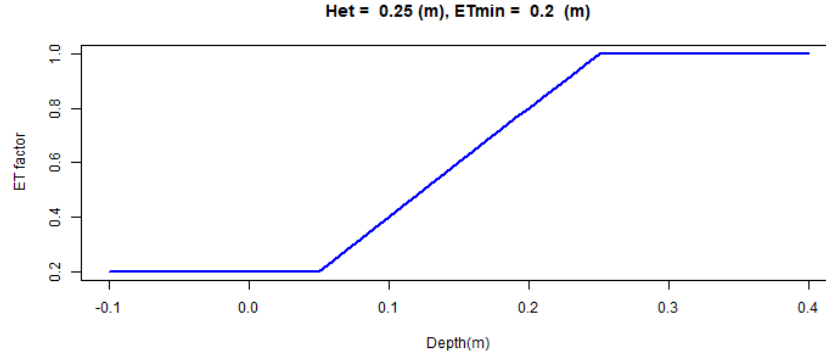


Figure 3.10: The dimensionless evapotranspiration reduction factor is a function of 2 parameters, Het and $ETmin$. The factor is calculated by the function $Fet(depth)$.

DBHYDRO for use in the earlier SRSM (Roth and Meselhe, 2010).

Actual ET may be less than potential ET because less water is available for ET when conditions are dry. This reduction in ET is modeled in function (function $Fet(d)$) as a dimensionless depth-dependent multiplier (Figure 3.10).

3.5 Temporal scale

Daily average values are used for input time series data for flow, rain depth, ET, and stage. Daily values are used because they are more generally available and because the use of more frequent data (hourly, for example) would greatly increase the size of the datasets. Although daily values are believed to have acceptable temporal resolution, future studies might examine model sensitivity to using more frequent data.

3.6 Model Structure

The model spatial structure is diagrammed in Figure 3.11. The model divides the Refuge into $ncell = 39$ cells and connects the cells with $nlink = 52$

links, providing flow paths between adjacent model cells.

3.7 Time, day, and date

The model simulates time as a continuous variable measured in days from a base time of midnight on the morning of January 1, 1995. Days are counted as integers, with day one corresponding to January 1, 1995. When the variable *time* = 0 the date and time are January 1, 1995 at 0:00 hours, and *time* = 1 is 24 hours later, January 2, 1995 at 0:00 hours. Thus, a model output for time equal to an integer *n* corresponds to the end of day *n* and the beginning of day *n* + 1, and the day corresponding to any time greater than zero is the integer part of time plus one. Three functions are included in the model to clarify this relationship in the R code.

```
Day2TIME(d)    function converts model day to time at the
                beginning of day
Date2Day(d)     function converts a date (type character or date)
                to day
Day2Date(d)     function converts model day to date (type date)
```

3.8 Parameters

The model parameters are constants used in the model calculations. Most model parameters are set in a separate R script that is selected at runtime by the user. The "base run" parameters are set in the script *39BoxBase.R*. This file should not be edited by the user. When running alternatives, a renamed copy of this file should be created, and only the renamed copy should be edited to make parameter changes. This filename should be entered when prompted by the *runStage.R* script. The filename is then used by the model to name the output file.

3.9 Timeseries and geometric data

A large amount of data is required to run the stage model. Although these data are generally available for the Refuge, the task of collecting, cleaning,

formatting, and saving all the data required to run the model is a large part of model development.

Time series data are required by the model to define historic conditions of flow, precipitation, and ET. Geometric data provide measurements for model cells and links.

3.9.1 Input data types and sources

Data for the current model version were obtained from spreadsheets used to build the Berkeley Madonna model, or exported directly from the Berkeley Madonna model. Spreadsheets used to create the current model dataset include:

CellParameters.xlsx - data transferred to dataframe "cell"

- "Area" - surface area of cells
- "Elevation"- cell ground surface elevation
- "MinVol" - minimum cell volume

LinkParameters.xlsx - data transferred to dataframe "link"

- "Radius" - radius of link
- "width" - width of link
- "linkupdn" - upstream and downstream link numbers

INFLOW39.xlsx - data transferred to dataframe "Inflow"

- "INFLOW" - historic inflows by structure

OUTFLOW39.xlsx - data transferred to dataframe "Outflow"

- "OUTFLOW" - historic outflow by structure

Regulation39.xlsx - data transferred to dataframe "NonReg"

- "Regulation" - non-regulatory outflows

PET39.xlsx - data transferred to dataframe "PET"

- "PET" - potential evapotranspiration

Volume-Stage-Canal.xlsx - data transferred to dataframe canalVS

"S-V" - canal cell stage-volume pairs

obs_stageall.xlsx - data transferred to dataframe "Stage.Obs"

"obs_stageall" - observed stage for calibration

3.9.2 R dataset creation

The script *make_datasets.R* imports the time series and geometric data. The script stores the compiled data in R data structures and saves these structures in the file *39-Boxtext-Datasets.Rdata*. This file is then loaded when running the model.

3.10 Model Implementation in R

This model version was developed in the R programming language using the RStudio programming environment. R is widely used and well-suited for data handling. R supports versatile graphics development and includes comprehensive statistical analysis. R has the disadvantage of being slower than alternative languages and modeling platforms, including Berkeley-Madonna, which was previously used as the modeling software platform. R is especially slow compared to compiled languages such as FORTRAN, C, and Modelica.

3.10.1 Rationale for porting to R

The previous version of the 39-Box Model was implemented in the Berkeley-Madonna simulation environment. Berkeley Madonna has several advantages. Berkeley Madonna provides a relatively simple and flexible user interface for writing algebraic and differential equations that describe the system being modeled. It also provides an interface for parameter optimization, sensitivity analysis, and graphic production.

However, Berkeley Madonna has some limitations. Although the model input for Berkeley Madonna may be shared as text files or in the proprietary *.mmd input files, the Berkeley Madonna simulation environment must be

purchased by users to run simulations. Berkeley Madonna is also not widely used. These limitations led to the decision to translate the 39-Box model into the R programming language.

R is a widely used open-source programming language. In addition to the core R language, many library packages are freely available to extend R's capabilities. R is an interpreted rather than a compiled computer language. This is an advantage in ease of use and program development, but because R is interpreted, execution of an R program can be orders of magnitude slower than a similar compiled program.

3.10.2 Key R program libraries used

Several packages must be installed before running the model. In R-Studio, packages are installed from the Tools tab. Required libraries are invoked in the "Preamble" section of the model code near the beginning of the program.

```
library(chron)      # date functions
library(stats)      # statistics functions
library(deSolve)    # ordinary differential equation solvers
library(lubridate)  # additional date functions
library(dplyr)      # data manipulation
library(readxl)     # read Excel worksheets
library(matrixStats)# colMin function
```

3.10.3 File structure and organization

The model is implemented in files within a parent directory named *39-Box*. Folders within the parent folder contain most of the model implementation.

Rscript - All script and function files

DataSets - Data including Excel and CSV files

Documentation - Model documentation and figures

maps - Various map files

Calibration - Files related to model calibration

Output - Model output

Some files in these folders were copied from previous model versions and are not needed to run the current R model version.

Model scripts and code files

39BoxStage.R - This is the main model script that is executed each time the stage model is run.

39BoxStageConstants.R - This script sets the model constants needed for the 39-Box Stage model. The script is executed each time the model is run. Separating the stage model parameters into a separate file allows the user to save alternative model run settings in unique files and then copy the desired file to *39BoxStageConstants.R* when needed for model execution. It also allows the user to save this script file along with model output for a specific model scenario run.

Day2Date.R - Contains the 39-Box day and date functions.

depth.R' - Contains functions to calculate cell depth and volume.

Main functions and their purposes

The use of functions simplifies code debugging and results in a more modular program that is easier to maintain and modify.

Day2Date - Converts model day to date

Date2Day - Converts date to model day

Day2TIME - Converts day to time

Cell.Depth.calc - Calculates cell depth from volume

Cell.Volume.calc - Calculates cell volume from depth

Canal.BF.calc - Calculate canal bank-full parameters

QoutHistoric - Historic canal outflows by cell on DAY

Qinfunc - Historic canal inflows by cell on DAY

A1Floor - Regulation Schedule A1 floor stage

QCalcOutS10 - Calculated regulatory release

QoutCalcCell - Distributes calculated outflows among canal cells

Fet j- Calculate ET depth reduction factor (dimensionless)

Link.Flow.calc - Calculate link flows

link2cell - Link flows added/subtracted to/from cells

derivs - Called by function ODE, returns d/dt cell volume

Output file

The model output filename is set in the *39BoxStageConstants.R* script using the constant *run.filename* and is assigned the file extension *Rdata*.

3.11 Model calibration

The previous version of the 39-Box model was calibrated and tested for stage and depth using USGS stage data (Figure 3.12) and depth measurements observed at water quality monitoring sites (Figure 3.13). Depth measurements in the Everglades are typically termed depth to consolidated sediment (DCS). This distinction is needed because there is often an observable thin layer of detrital floc covering the sediment surface.

Parameters related to water transport were also calibrated by comparing monitored chloride concentration monitoring data with simulated chloride concentration. At the time of this writing, there has been no effort to calibrate the new translated model. In the future, a new model calibration study could be conducted using data extending beyond 2009 and employing the chloride model (39-Box Mass Model), which is currently in development.

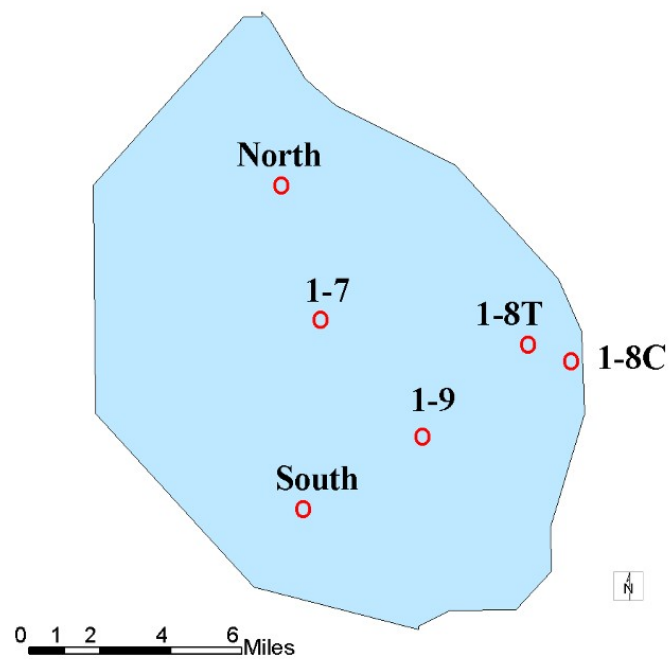


Figure 3.12: Location of USGS stage gages (Bazgirkhoob *et al.*, 2015).

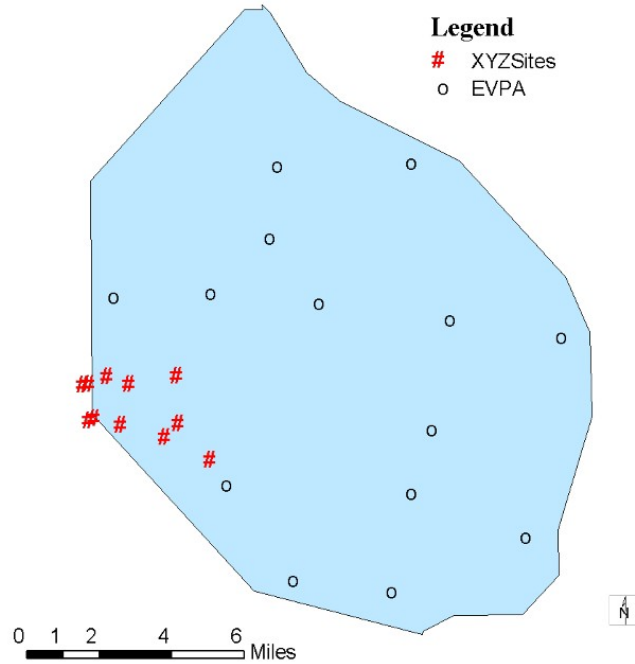


Figure 3.13: EVPA and XYZ water quality monitoring sites provide water depth observations that were used to calibrate the previous version of the 39-Box model (Bazgirkhoob *et al.*, 2015).

Chapter 4

Running the Model

4.1 Model package description

The 39-Box stage model package is a collection of data and script files. Some scenarios will involve only a simple change to a model parameter. However, many of the possible scenarios that users may wish to study using the model will require an understanding of the data and R scripts used during a model run.

The data used in the 39-Box simulation were first compiled into an R dataset file using the script *make_dataset.R*. Excel spreadsheets are read by the script *make_datasets.R* into the model data frames that define much of the model temporal boundaries, topography, and structure. Some of these spreadsheets were used to develop previous model versions. In other cases, data were exported from the earlier SRSM model (Roth, *et al.*, 2010) into spreadsheets. Datasets and some constants are saved in an R data file *39-Box-Datasets.Rdata*.

There are several R scripts included in the model package, including:

- *runStage.R* This script initiates the model run. The script sets the file name of the model constants script as the default, *39BoxBase.R*,

or as a user entered filename. It then runs the chosen constants script followed by *39BoxStage.R*, and then *39BoxStageStdGraphs.R*.

- *39BoxBase.R* Default model constants are set in this script. To avoid confusion, users are urged not to edit this script. Instead, when running a scenario that requires changing constants, users are advised to edit the required changes in a copy with a new descriptive name. Then, enter this filename when prompted.
- *39BoxStage.R* This script executes the simulation.
- *39BoxFunctions.R* Model functions are defined in this script. It is sourced by *39BoxStage.R* at the beginning of the model run.
- *39BoxStageStdGraphs.R* Plots a number of standard graphs at the end of the model run.
- *WaterBalance.R* Runs an analysis of the model water balance.

All model script files are self-documented using comments that provide additional details that are not described in this manual.

4.2 System requirements and installation

No detailed information is available on computer memory and storage requirements. Memory usage in RStudio is displayed in the small memory usage widget in the Environment pane. The widget pie chart displays the total amount of system memory in use, and the number displays the approximate amount of memory in use by the R objects, libraries, and packages. The color in the widget changes to red when you run out of memory.

However, often it will not be necessary to create a new model directory.

4.3 Output

Model results are saved after a model run in an R dataset in the Output folder. The output filename is set to a user-specified filename (variable

name *filename*) at the beginning of the run. The output file has the extension *.Rdata*. This file is created by these lines of code at the end of the *39BoxStage.R* script.

```
# Save output
save(run.title, filename, sim.Volume, sim.Depth, sim.Stage,
     sim.Outflow, sim.Linkflow, sim.Velocity,
     file=paste("../Output/",filename,".Rdata", sep=""))
```

- *run.title*, and *filename* are the user specified title used in graphs, and the input/output file name.
- *sim.Volume*, *sim.Depth*, and *sim.Stage* are matrices of daily values (rows) for all cells (columns).
- *sim.Outflow* is a matrix of daily outflow (row) for all canal cells.
- *sim.Linkflow*, and *sim.Velocity* are matrices of daily values (row) of discharge and velocity for all links (columns).

The values in the rows of the output matrices are instantaneous values at the start of the day. The first row of the output matrices holds the initial values at time 0:00 on the first day of simulation. In general, output matrix row n holds the value at the beginning of day n and row $n + 1$ holds the value at the end of day n . Daily average values are not calculated by the model. However, the daily average value for day n may be approximated as the average value for rows n and $n + 1$.

4.4 Axillary Scripts for Output Visualization

4.4.1 Particle Tracking

Particle tracking is a widely used method for visualizing and analyzing water movement in two-dimensional models. Particle tracking simulates the release of small neutrally-buoyant particles. The path of each particle is then calculated from the simulated water flow velocities. This provides an

intuitive representation of the transport pathways, residence times, and flow patterns.

The particle-tracking script *Tracker.R* displays the path of a single simulated particle. The script displays simulated tracks through the Refuge marsh and canal until the particle exits through an outflow. Particle movement is calculated using previously modeled link velocities stored in the matrix *sim.velocities*. Particle movement through dispersion, water loss to groundwater seepage, and loss through evapotranspiration are ignored.

The tracking script first asks the user to enter the starting date and cell number. The simulated particle moves from a cell into an outward flowing link based on a probability weighted by the magnitude of the link outflow (function *cell.drop*). The particle is then moved along the link (function *link.route*) at the daily link velocity until it exits the link and moves into a cell.

After a tracked particle exits, the user is asked to enter a new cell number to initiate a new track. Users can select the same cell number multiple times to visualize the multiple flow paths that may be followed from a single starting cell.

4.4.2 Plotting contours of marsh properties

The function *MarshMap.contour* creates contour maps of the variables associated with the model marsh cells. The function script is stored in the file *MarshMapcontour.R*. The function has a single required argument *z*, which is a vector of length *ncell* = 39 values associated with all model cells. Currently, only the values associated with marsh cells are used to generate the contour plot. The function also has several optional arguments that can be specified to change the plot appearance.

As a simple example, a contour plot of marsh elevation used in the model can be generated by typing the following command in the console window (Figure 4.1):

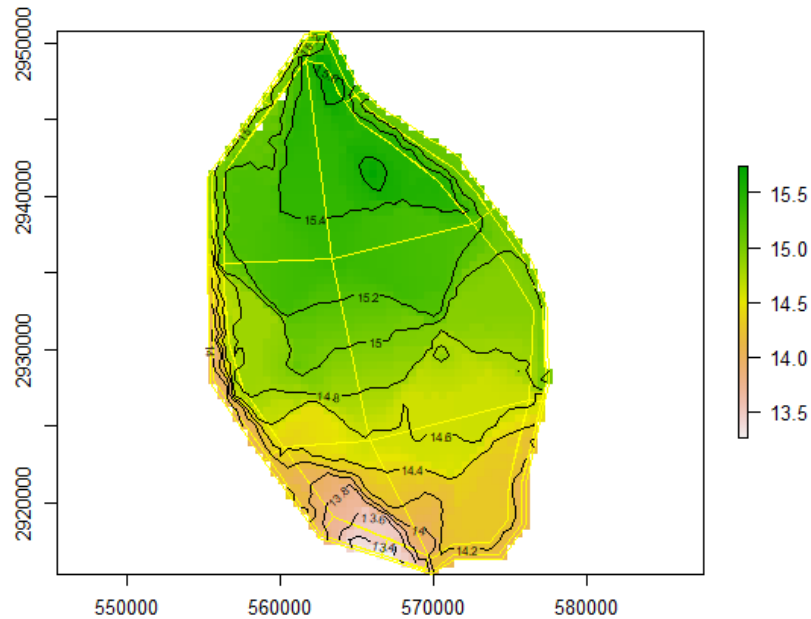


Figure 4.1: Marsh soil elevation converted from meters (NGVD29) to feet is plotted using default settings in function *MarshMap.contour*.

```
MarshMap.contour(cell$E0/0.3048)
```

Division by 0.3048 m/ft is used here to convert soil elevation from meters to the conventional units of feet. The function returns a named list containing a spatial points dataframe named *xyz*, which contains all points used to generate the plot. Optionally, if the argument *plot.return = TRUE* is provided, the returned list additionally contains the plot object *cmap*.

The optional arguments and their default values are listed below.

```
cplot.date =    '', date to be included in plot title
cplot.title =   '', plot title
raster.length = 500, raster length in meters
idp =           1.0, inverse distance interpolation parameter
zlimit =        NULL, min and max z values
```

```

UseCentroid = FALSE, use polygon centroid, not plotting point
boundary.add = TRUE, additionally use points on polygon borders
plot.do =     TRUE, draw the plot in a window
plot.return = FALSE, return the plot as an object

```

The argument *cplot.date* may be either a date object or text compatible with conversion to a date, and *cplot.title* is a text string displayed at the top of the plot, followed by the date if specified.

The script first collects a set of xyz data points into a data frame by combining each marsh value in the *z* argument with a coordinate xy pair for the corresponding polygon. Because of the curved shape of cells near the canal, cell polygon centroids may not be located in the interior of the polygon. By default, plotting points for the polygon are placed at coordinates that fall within the cell polygon. The plotting point values are calculated by the function *point_in_cell* using the *sf* library function *st_point_on_surface*. These plotting coordinate points are added to the *cell.plotxy* matrix by the function *sf.read*, and are optionally returned by the function *cell.xy*. The optional argument *UseCentroid = TRUE* specifies that the polygon centroid coordinates are used instead of the plotting point.

If *boundary.add = FALSE* then the raster values used to generate the contours are interpolated from only the marsh values specified in the *z* argument. The cluster analysis, which was the basis for drawing the cell boundary polygons, supports the conjecture that values at the cell boundaries should be near the mean of their bordering cells. By default (*boundary.add = TRUE*), function *z_points.add* is called to add values at all cell polygon vertices. These added points are set to the mean of the values for all bordering cells. When the boundary vertices are far apart relative to the *raster.length*, additional boundary values are interpolated from the adjacent polygon vertex points (function *ls.add*).

Next, the script creates a spatial raster object of the values. This raster is then used to interpolate the map contours. Two optional arguments control the construction of raster points. The optional argument *raster.length*

specifies the length of the raster grid cells, and the optional argument *idp* is a parameter used by the inverse distance weighting model to interpolate the discrete spatial data onto the raster grid.

The span of values displayed by the contours is set optionally by the argument *xlimit*. For example, these limits the argument *xlimit* = *c*(4.0,6.0) sets the minimum and maximum contour values to 4 and 6, respectively. The function *cell.xy* is called to determine the xy-plotting coordinates of a single point for each cell polygon.

By default, a contour plot is created and displayed in the window. If *plot.do* = *FALSE* the plot is not created. A plot object may be included as an item in the returned list by specifying the argument *plot.return* = *TRUE*.

A more complex example plots the simulated marsh depth on January 1, 1999, setting the raster length and scale limits for the contours (Figure 4.2).

```
MarshMap.contour(sim.Depth[Date2Day('1999 - 01 - 01'),], zlimit =  
c(0,1.4),raster.length = 250), cplot.date = '1999 - 01 - 01', cplot.title =  
Simulated Marsh Depth (m)')
```

4.4.3 Plotting canal properties

The function *Canal_Map* creates a color (choropleth) map of a variable associated with the model canal cells. The function script is stored in the file *Canal_Map.R*. The function has a single required argument *z*, which is a vector with a minimum length *ncanal* = 11 values associated with all canal cells. The function also has several optional arguments that can be specified to change the plot appearance.

The canal width is very small relative to the extent of the Refuge coordinates. Therefore, the function redraws the canal color map several times, each with a small offset added to all xy coordinates. Because of this re-plotting, the displayed canal width is not plotted to scale.

As a simple example, a color plot of the canal bottom elevation used in the

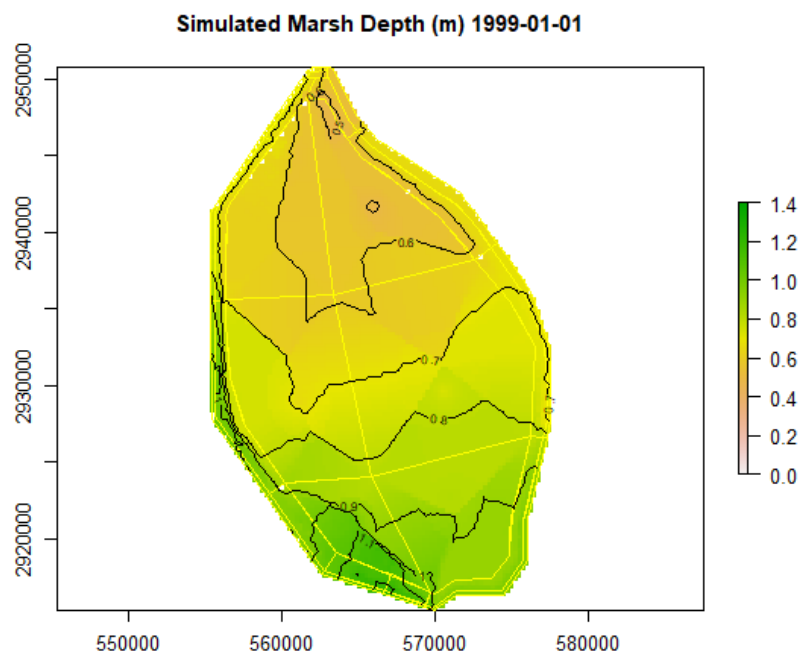


Figure 4.2: Simulated marsh water depth (m) on January 1, 1999 plotted by function *MarshMap.contour*

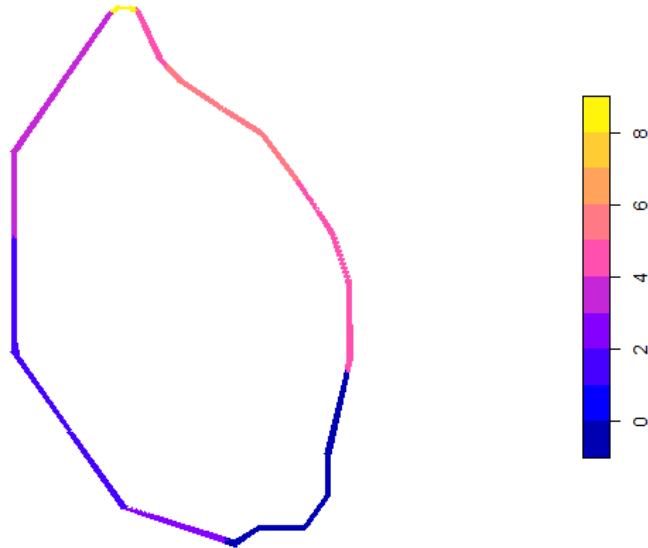


Figure 4.3: Canal minimum or bottom elevation converted from meters (NGVD29) to feet is plotted using default settings in function *Canal_Map*. Note that canal width is exaggerated.

model can be generated by typing the following command in the console window (Figure 4.3):

```
Canal_Map(cell$E0/0.3048)
```

Division by 0.3048 m/ft is used here to convert the canal minimum elevation from meters to the conventional units of feet.

Upon successful completion, the function returns the value *TRUE*.

The optional arguments and their default values are listed below.

```
plot.date =    '', date to be included in plot title
xy.step = 40, additional offset used in overlaid plots
n=steps = 10, number of plots to overlay
zlim = NULL, numeric vector of min and max z
ncolor = 11, number of colors used in palette
axes = FALSE, if TRUE then draw xy axes
```

```
pal = NULL, color palette
```

The argument *plot.date* may be either a date object or text compatible with conversion to a date, and *plot.title* is a text string displayed at the top of the plot, followed by the date, if specified.

The span of values displayed by the colors is set optionally by the argument *zlim*. For example, the argument *zlim = c(-2.0, 10.0)* sets the minimum and maximum *z* values to -2 and 10, respectively.

The option *pal* sets the color palette used in the color plot. The use of this option can produce errors. Users are currently advised against using the *pal* option.

An example of a plot using some of the options is shown below, and the resulting plot is displayed in Figure 4.4.

```
d <- '2007-07-15'
```

```
Canal_Map(sim.Depth[sim.i(d),], n.steps = 20, xy.step = 40,
```

```
plot.date = d, plot.title = 'CanalDepth', axes = TRUE)
```

The function *sim.i(d)* is used here to convert the text date, *d*, to the corresponding row index of the simulation output matrix *sim.Depth*.

4.4.4 Additional location-related functions

The 'LocationFunctions.R' script contains R functions that were translated from VBA macro routines found in the Excel workbook **RefugeSiteXY values.XLS**. This workbook compiles site locations for Refuge monitoring sites and performs coordinate conversions and other location-related calculations. The coordinates used in these functions are in **NAD83 UTM Zone 17 North** for X and Y values and **WGS 84** for Latitude and Longitude in decimal degrees.

The functions in *LocationFunctions.R* are categorized into three main groups:

Coordinate Conversion Functions (Translated from VBA) These

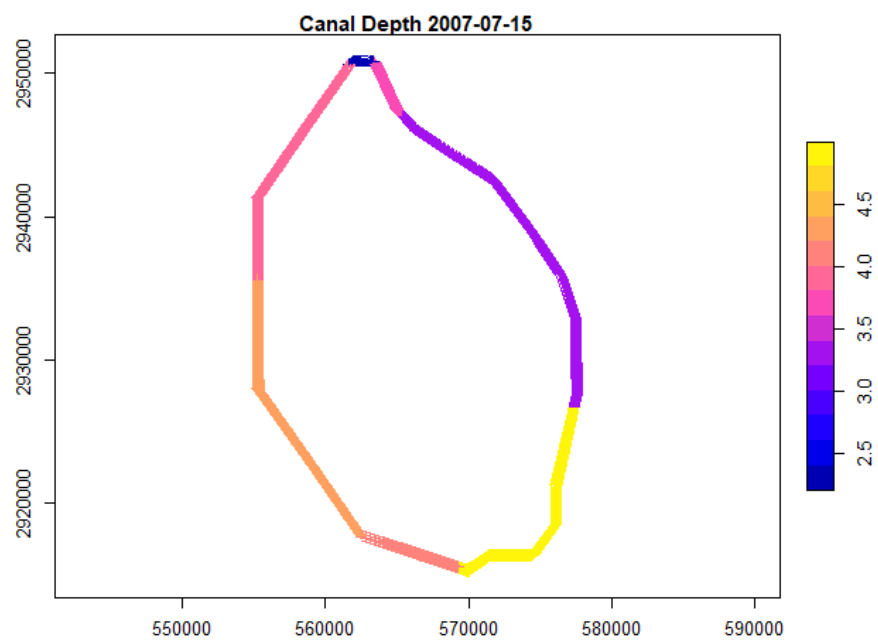


Figure 4.4: Simulated canal depth (m) on July 7, 2007, is plotted using function *Canal_Map* to illustrate the use of some of the function options. Note that canal width is exaggerated.

functions facilitate the conversion between Latitude/Longitude and UTM X, Y coordinates:

- **XCalculate(Lat, Lon)**: This function returns the **X coordinate** (UTM) from given Latitude (**Lat**) and Longitude (**Lon**) values, both in decimal degrees.
- **YCalculate(Lat, Lon)**: This function returns the **Y coordinate** (UTM) from given Latitude (**Lat**) and Longitude (**Lon**) values, both in decimal degrees.
- **LongCalculate(X, Y)**: This function returns the **Longitude** (decimal degrees) from given UTM X and Y coordinates.
- **LatCalculate(X, Y)**: This function returns the **Latitude** (decimal degrees) from given UTM X and Y coordinates.

Geometric and Distance Calculation Functions (Translated from VBA) This group of functions focuses on calculating distances and properties related to lines and segments defining the Refuge boundary and canal.

- **CanalMeters(X, Y, SegTable)**: This function calculates the **clockwise canal distance in meters** from a specified point (**X, Y**) to the closest point on the canal. The canal distance measurement starts at the **G-300** control structure and proceeds clockwise around the Refuge canals.
- **CanalDistance(X, Y, SegTable)**: This function returns the **shortest distance** from a given point (**X, Y**) to the closest segment of the canal boundary.
- **NearestSeg(X, Y, SegTable)**: This function identifies and returns the **index of the segment closest** to the given point (**X, Y**).
- **PointLineCalc(PtX, PtY, X1, Y1, X2, Y2)**: This is a core utility function that calculates the **coordinates (xmin, ymin)** on a line segment that are perpendicular to an off-line point (**PtX, PtY**). It also returns the **distance (PointLineDist)** from the point to the line.

A negative distance indicates that the perpendicular intersection is outside the segment. It handles special cases, such as identical points or perfectly vertical/horizontal lines.

- **PointSegDistance(PtX, PtY, X1, Y1, X2, Y2)**: This function returns the **distance from a point** (PtX, PtY) to a **line segment** defined by its endpoints (X1, Y1, X2, Y2). If the perpendicular projection of the point falls outside the segment, the function returns the distance to the nearest segment vertex.
- **PointLineDistance(PtX, PtY, X1, Y1, X2, Y2)**: This function calculates the **distance from a point** (PtX, PtY) to a **line** (not necessarily segment-bounded) defined by two points (X1, Y1, X2, Y2). It can return a negative value if the closest point on the line is outside the segment defined by X1, Y1, X2, Y2.
- **PointLineX(PtX, PtY, X1, Y1, X2, Y2)**: This function returns the **X-coordinate** of the point on a line (defined by X1, Y1, X2, Y2) that is perpendicular to a given point (PtX, PtY). If the perpendicular intersection is outside the segment, it returns the x-coordinate of the nearest vertex.
- **PointLineY(PtX, PtY, X1, Y1, X2, Y2)**: This function returns the **Y-coordinate** of the point on a line (defined by X1, Y1, X2, Y2) that is perpendicular to a given point (PtX, PtY). Similar to **PointLineX**, if the perpendicular intersection is outside the segment, it returns the Y-coordinate of the nearest vertex.
- **SegLength(X1, Y1, X2, Y2)**: This function calculates and returns the **Euclidean distance** (length) between two points (X1, Y1 and X2, Y2) which define the ends of a line segment.

Other Useful Functions These functions provide additional utilities for working with spatial data and the compartmental structure of the model:

- **SegTable.create()**: This function loads the **Refuge boundary segment table** from the `Refuge Site X Y values.XLS` spreadsheet into

an R data frame. It includes information such as segment IDs, vertex coordinates, and lengths.

- `Refuge.xybound(SegTable)`: This function takes the segment table as input and returns the **bounding box coordinates** (xmin, xmax, ymin, ymax) that encompass the entire Refuge area.
- `is.in.polygon(x, y, pol_x, pol_y)`: This function determines whether a single point (x, y) lies **inside or on** a polygon defined by its X and Y vertex coordinates (pol_x, pol_y). It returns a logical TRUE if the point is within or on the boundary of the polygon.
- `Cells_in(x, y, dxy = 25, cell.list = NULL)`: This function identifies and returns a **vector of cell numbers** that contain the specified point (x, y). It can also check adjacent locations based on the `dxy` resolution distance.

4.5 Workflow

There is no fixed set of steps required to analyze a new model scenario. In this section, some general guidelines for scenario analysis are suggested, and an example scenario is presented.

When running a scenario, it is important to avoid inadvertently changing the distributed base model files. Changing these files could result in unknowingly modifying other scenario analyses. If changes to distributed model files cannot be avoided, then all model files should be copied to a new folder dedicated to that scenario.

As an example, consider a scenario concerning the date on which the S-6 pump was diverted. Historically, the S-6 pump discharged into the perimeter canal in the southwest of the Refuge. This inflow was diverted to WCA-2 in 2001. In the model flow record, the last day of inflow from the S-6 pump was March 30, 2001. In this example scenario, consider how this diversion changed marsh stage by removing this inflow beginning before 1995. A comparison of the refuge hydrological response to the S-6 diversion

was performed by comparing the period 1995 to 2001 with and without S-6 inflows.

The files required for this scenario are included in the distributed model. Note that users are advised not to edit the original files in the distribution. To create this scenario, the file *runStage_NoS6inflow* was copied from the distributed file *runStage.R*. Two sections of the new file were edited, as shown below.

First section sets inflow from the S-6 to zero:

```
# -----
# On the following lines make any changes needed for the
#   current scenario.
#   This may alter constants, loaded dataframes, or redefine
#   functions.
# Example: To model the impact of increasing flow through
#   the S39 by 20%
#   NonReg$S39 <- 1.2*NonReg$S39
# Save this modified file with a new filename (i.e., not
#   as runStage.R).

# set the S-6 inflow to zero
Inflow$S6 <- 0
# set the end date of the simulation to March 30, 2001
# This is the last day of historic S-6 flow > 0.
Stop.Date <- as.Date('2001-03-30')

# -----
```

Second section calls a new graphics script:

```
# -----
# On the following lines add any final statistical calculations,
#   or graphics

# run a graphics script
```

```
source("S6ScenarioGraph.R")
```

The file script was then executed (sourced), and at the user prompt name the output file *NoS6Inflow* was entered. This created a new output file *NoS6Inflow.rdata*, produced 2 graphics (Figures 4.5 and 4.6). It also printed the result that the hydroperiod of interior marsh cell 36 changed from having an average of 0.6 days per year with depth below a threshold of 0.1 m to 16.2 days below this threshold.

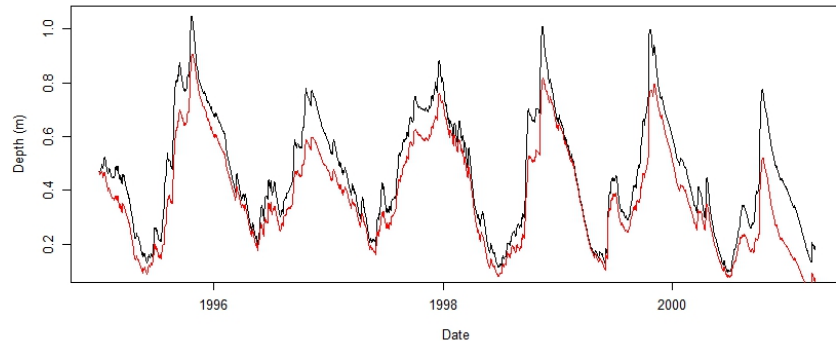


Figure 4.5: Depth in cell 36 with (black) and without (red) inflow from the S-6 pump.

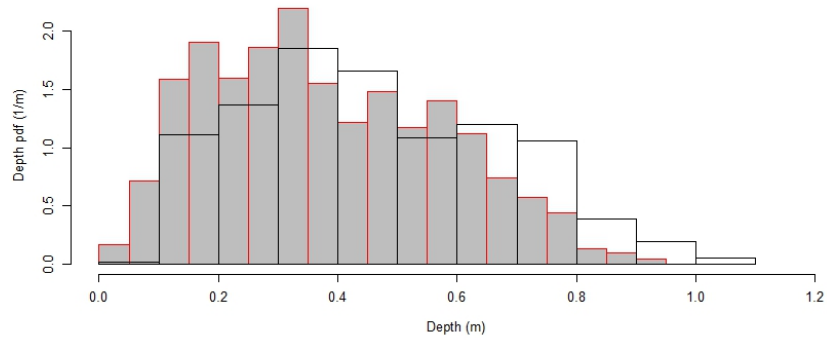


Figure 4.6: Histogram of depth in cell 36 with (black) and without (red) inflow from the S-6 pump.

Chapter 5

References

Arceneaux, J., E. A. Meselhe, A. Griborio, and M. G. Waldon. 2007. The Arthur R. Marshall Loxahatchee National Wildlife Refuge Water Budget and Water Quality Models. Lafayette, Louisiana: University of Louisiana at Lafayette in cooperation with the U.S. Fish and Wildlife Service.

Bazgirkhoob, Hamid, Ehab A. Meselhe, and Michael G. Waldon. 2015. A.R.M. Loxahatchee National Wildlife Refuge 39-Box Model User's Manual (revised). Prepared for the U.S. Fish and Wildlife Service under a cooperative agreement with the University of Louisiana at Lafayette.

Chen, C., E. A. Meselhe, and M. G. Waldon. 2009. A.R.M. Loxahatchee National Wildlife Refuge Hydrodynamic Modeling with MIKE FLOOD (Version 2.0). Prepared for the U.S. Fish and Wildlife Service.

German, E. R. 2000. Regional evaluation of evapotranspiration in the Everglades. Water-Resources Investigations Report 2000-4217. U.S. Department of the Interior, U.S. Geological Survey.

Macey, R., Oster, G., Zahnley, T., 2000. Berkeley Madonna User's Guide Version 8.0. University of California, Department of Molecular and Cellular Biology, Berkeley, CA.

Meselhe, E., Arceneaux, J., and Waldon, M. G. (2007a). Mass Balance

Model Version 1.01. Report #LOXA-07-002, University of Louisiana Lafayette, in cooperation with the US Fish and Wildlife Service, Lafayette, LA.

Meselhe, E., Arceneaux, J., Waldon, M. G., and Wang, H. (2007b). A.R.M. Loxahatchee National Wildlife Refuge Water Quality Model Version 1.00. Report #LOXA-07-003, University of Louisiana Lafayette in cooperation with the US Fish and Wildlife Service, Lafayette, LA.

Meselhe, E., Waldon, M. G., and Roth, W. 2009. A.R.M. Loxahatchee National Wildlife Refuge Simple Refuge Screening Model Version 4.00 User's Manual. Lafayette, Louisiana: University of Louisiana - Lafayette.

Neidrauer. 2004. Water Conservation Area Regulation Schedules. Presented at the Technical Oversight Committee Meeting, August 2004.

Richardson, J. R., W. L. Bryant, W. M. Kitchens, J. E. Mattson, and K. R. Pope. 1990. An evaluation of refuge habitats and relationships to water quality, quantity, and hydroperiod: A synthesis report. 166 pages + map, Cooperative Fish and Wildlife Research Unit, University of Florida, Gainesville, FL.

Richardson, J. R., W. L. Bryant, and W. M. Kitchens. 1993. A spatial hydrologic model of Loxahatchee National Wildlife Refuge and its integration into a GIS analysis. Pages 450-454, Wetlands: Proceedings of the 13th Annual Conference, New Orleans, Louisiana, June 1992. M. C. Landin, ed., Society of Wetland Scientists, South Central Chapter.

Roth, W. B., and E. A. Meselhe. 2010. The Development of a screening model for the Arthur R. Marshall Loxahatchee National Wildlife Refuge. 189 pages. Lafayette, Louisiana: Prepared for the U.S. Fish and Wildlife Service, Department of the Interior, by the Institute of Coastal Ecology and Engineering, University of Louisiana-Lafayette.

USFWS. 2000. Arthur R. Marshall Loxahatchee National Wildlife Refuge Comprehensive Conservation Plan. 362 pages. Boynton Beach, Florida: US Fish and Wildlife Service.

Wang, H., E. A. Meselhe, M. G. Waldon, D. D. Surratt, S. Abdou, C. Chen, and M. C. Harwell. 2008. Compartment delineation for a wetland water quality model in the Northern Everglades, Florida, USA. *Journal of Environmental Hydrology*, 16: Paper 36.

Wang, H., E. A. Meselhe, M. G. Waldon, M. C. Harwell, and C. Chen. 2012. Compartment-based hydrodynamics and water quality modeling of a Northern Everglades Wetland, Florida, USA. *Ecological Modelling*, 247: 273–285.

Appendix A

Glossary of Terms

Cell - may also be termed a **box**, **compartment**, **tank**, or **node**. A cell covers a specified contiguous area, and all cells cover the entire area of the model domain. A cell is an object with the properties of type (canal or marsh), area, volume, soil elevation, average depth, and stage.

Depth - Height of the water column above the consolidated marsh soil surface (see DCS).

DCS - depth to consolidated sediment (DCS) is the depth measurement used in Everglades monitoring programs.

Elevation - Height above a specified datum. All elevations in the 39-Box model are relative to the NGVD29 datum.

Link - connects adjacent cells with a flow path. A link is an object with properties of type (canal-canal, canal-marsh, marsh-marsh), upstream cell, downstream cell, width, radius (or length), discharge, and velocity.

Object - a programming concept that has properties and methods. Here, objects are primarily defined using data frames with properties assigned in the data frame definition (for example `cell$type`), and methods are defined using functions (for example, the function `Cell.Depth.calc(v)` returns cell depths given their volumes).

Refuge - The Arthur R. Marshall Loxahatchee National Wildlife Refuge in Florida, USA.

Regulation schedule - A water control plan for managing outflow from the Refuge.

SFWMD - South Florida Water Management District

STA - Stormwater Treatment Area, a constructed wetland treatment system.

Stage - The water surface elevation above a specified datum surface.

WCA - Water conservation area

Appendix B

Extending the model timeseries

The model timeseries data are described in section 3.9. The Berkeley-Madonna version of the 39-Box model ran from January 1, 1995, to June 30, 2009. Detailed descriptions of compiling these data are provided in older model documentation, including:

(list of documents to be added)

The model now includes updated datasets that extend the input data until June 30, 2025. The new data were downloaded from the South Florida Water Management District data portal, DBHYDRO insights.

B.1 Evapotranspiration (ET)

Daily evapotranspiration estimates were downloaded from DBHYDRO insights for the LOXWS site located on the L-40 Canal at the 1-8C gage. This ET timeseries corresponds to DBHYDRO key (DBKEY) RW485. The Excel workbook *ET.xlsx* compiles the daily potential ET calculated at the LOX weather station (DBKEY RW485). A final timeseries is compiled on sheet *ET* using RW485 ET data from July 1, 2009, through April 30, 2025.

B.2 Rainfall

Total daily rainfall data were downloaded from DBHYDRO insights for rain sites at pump station S-6, the S-39 gate, pump station S-5A, site LOXWS located on the L-40 Canal at the 1-8C gage, and for two sites in STA1W named STA1W_R and G309. These series correspond to DBHYDRO keys (DBKEYS) 15203, VN276, 15202, VN116, DU517, KN809, and AM119, respectively. The two series for STA1W were combined into a single series that preferentially uses data from G309, and uses the STA1W_R values when the preferred values are missing. These 5 time series had no missing daily values over the period from July 1, 2009, through June 30, 2025.

Weighted daily averages of the 5 time series were calculated in the Excel workbook *Rain.xlsx*. Weights were determined in script *RainSiteArea.calc.R* from areas of Voronoi polygons clipped to the Refuge boundary (Figure B.1).

B.3 USGS daily average stage data

Data from the six USGS stage gages in the Refuge, from July 1, 2009, through September 30, 2025, were downloaded from DBHYDRO in the NGVD 29 datum in the file *USGS_Stage_Update.csv*.

The function *USGS_Stages.read* in file *39BoxFunctions.R* was revised to read the updated data csv file and combine the new stage observations with the older time-series. All values for the North and South gages were missing in this download because the data were only available in the NAVD 88 datum.

The North and South gages require special attention. Because of budget constraints at the time of installation, the NGVD 29 datum could not be determined through surveying, and the North and South gages were set to an arbitrary datum. Through comparison with the 1-8C gage, it was determined that a correction of 0.48 m had to be added to the North and South readings to correct the datum error to approximate the NGVD 29 stage.

Voronoi Polygons Clipped to Boundary

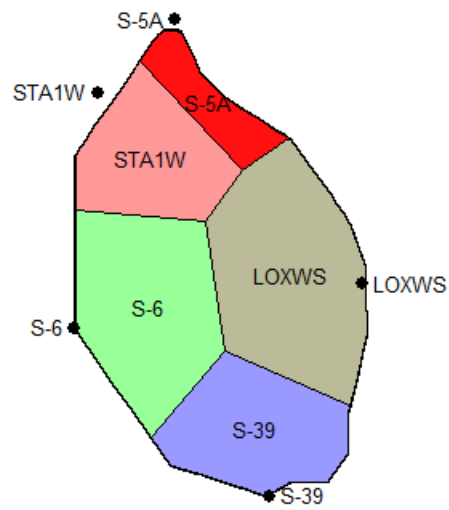


Figure B.1: Daily rainfall sites (black circles) and Voronoi polygons used in the rain dataset extension (figure created by script *RainSiteArea_calc.R*).

Today, stage is only available for the North and South gages relative to the NAVD 88 datum, and a published offset to NGVD 29 has not been found. To obtain this offset, daily average stages from the North and South gages were compared to the 1-8C stage (*USGS_Stage_Update.xlsx* sheet *Analysis*). These data were filtered to include only days when the 1-8C stage was equal to or greater than 17.00 feet and was unchanged from the previous day. The offset to add to the NGVD 88 stage to estimate the NGVD 29 stage was found to be 1.610 ± 0.0038 and 1.617 ± 0.0025 (*mean* $\pm$ *S.E.*) feet for the North and South gages, respectively. A value of 1.61 ft (0.491 m) was selected for both sites.

Data for some dates were missing in each downloaded time series. This is not a problem for model calibration because the missing days can be excluded from the analysis. However, stage data at the USGS 1-8C site are needed for all simulated days when there is outflow at the S-39 gate to distinguish regulatory releases from water supply flows. Over the 11,139 days downloaded from 1995 to 2009, there were 152 missing daily values in the downloaded 1-8C time-series. Missing values in the downloaded 1-8C time-series were estimated by creating a new time-series *1-8Cfilled* in the Excel spreadsheet using the following sequence of estimates: (1) Missing values were estimated as the average of the previous and next day stages. (2) If only one of these days was missing, the non-missing value was used. (3) If both adjacent days were missing, a similar procedure was used with the values from two days before and after the missing point. (4) Finally, if no value was available, the minimum stage at the other downloaded gages was used as the estimate. This procedure filled in all the missing values in the 1-8C time-series. The filled 1-8C time-series is shown in Figure B.2.

B.4 Inflow/Outflow

Inflow/outflow daily average discharge timeseries were downloaded from the DBHYDRO database using an R script in the file *make_Q_update.R*. That script calls functions stored in the file *flowFormatter.R*, which was origi-

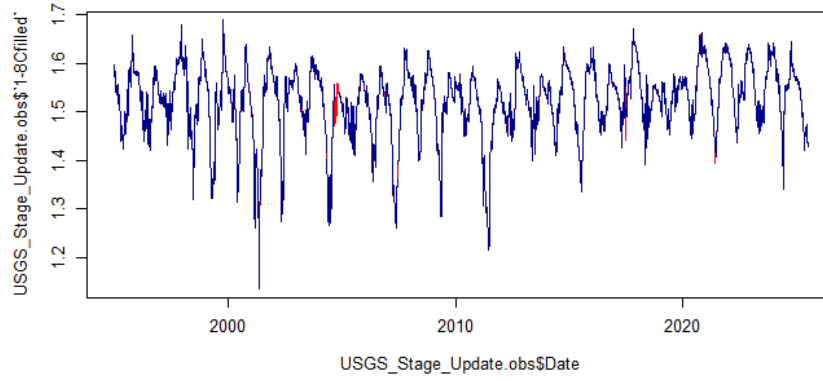


Figure B.2: Updated stage (m) time-series at gage 1-8C is plotted versus date. Values that fill in the missing data in the download are plotted in red.

nally provided by Dr. Donatto Surratt from the Everglades National Park. During execution, the R script places the downloaded flow data for each structure into a dataframe. All missing daily average flows were set to zero in the dataframe. That final dataframe and other information from the download are stored in a folder `../39BoxProject/DataSets/Qupdate` as binary R data files. The flow data for the update are also exported to a comma delimited `.csv` file named `Qdataupdate.csv`. Detailed information on the updated flow data can be found in the source file comments.